

Outbound Healthcare Voice Agent

Narration script for a screen-shared walkthrough.

Read the plain paragraphs aloud. **Boxed italic lines are stage directions for you, not for the viewer.**

About 12 minutes of narration plus a 3 minute live call. Do not rush the demo.

Before you press record: close the .env and .env.local tabs, close the Twilio and ElevenLabs consoles, and make sure the agent worker is already running.

PART 2 — How it's put together

Screen: [README.md](#), scrolled to the architecture diagram.

Two processes. `dispatch_call.py` starts a call. `agent.py` is a worker that connects to LiveKit and waits for work.

The dispatcher doesn't talk to the worker directly. It creates a room, then creates an agent dispatch with the patient record attached as metadata. LiveKit routes it. So the agent knows who it's calling and which biomarkers to discuss before the phone rings, and the two halves stay decoupled.

Scroll to the "three lines that wire in Opik" block.

And the observability is these three lines. Start a trace, attach to the session, finalise on shutdown. Nothing else in the codebase imports Opik. The brief asked for a standalone module and I'll show you that's literally true.

PART 3 — The prompt

Screen: `agent.py`, scrolled to `build_instructions`.

I'll start with the prompt, because it's the highest-leverage file here and it took four versions to get right.

Version one said "keep turns short." The agent ignored it. On a real call it delivered all three biomarkers in one unbroken paragraph and my tester interrupted with "wait, wait, wait, let me complete."

Point at the "TWO RULES THAT MATTER MOST" block.

So it became a hard limit. Two sentences, stated as a limit, not a preference.

Scroll through the numbered stages.

Then six explicit stages with a STOP after each. Greeting. Permission. Soften. One result. The appointment. Close.

The softening stage is the one I'd defend hardest. The agent may not say a number until it has first said something like "most of it came back fine, there's just one thing the doctor would like to look at." Opening with "your HbA1c is seven point eight" to someone who doesn't know why you're calling is a genuinely unpleasant thing to do to a person.

Point at the banned words list.

There's a list of words it may never say. Diabetes, diabetic, prediabetes, because labelling a condition is a clinician's job. And "does that make sense", because it's condescending and the model kept reaching for it.

Scroll to the worked example dialogue.

And this is what actually fixed the tone. Rules alone weren't enough. A short example conversation moved the register in a way more rules did not. Showing a model what good looks like beats telling it.

Screen: `data/patients.json`, point at the `plain_language` field on `HbA1c`.

One related thing. The first version of this data said "which is in the diabetes range", and the agent read it out word for word. The tone of a voice agent doesn't live only in the prompt. It lives in the data the prompt is grounded on.

PART 4 — Three bugs worth showing

Screen: `agent.py`, scrolled to the `greet` method.

Three bugs taught me more than the features did.

First. The greeting is not in `on_enter`, where you'd naturally put it. The session has to start before you dial, or you miss whatever the person says on pickup. But that means `on_enter` fires while the phone is still ringing.

So the agent was greeting a ringing line. The metrics showed eleven seconds of playback latency, and my tester answered midway through "Hello, this is Riley", heard a fragment, and the agent had nothing left to say. It now greets after the participant actually joins.

Screen: `post_call_analysis.py`, scrolled to `coerce_tool_output`.

Second, and this is the instructive one. Whether an appointment was booked is read from the tool record, not inferred by a model. I documented that as "authoritative, it cannot hallucinate."

It was quietly wrong. LiveKit serialises tool return values with `str()`, so a dictionary comes back as a Python repr with single quotes, and I was parsing it as JSON. Every successful booking collapsed into an opaque string and the answer was permanently false.

I only found it because I checked whether any real call had actually booked. One had. It booked twice and the analysis said no. Nothing crashed, nothing logged an error.

Screen: `opik_integration.py`, scrolled to `_write_spans_and_close`.

Third. I was closing each Opik trace with `trace.end()`. The SDK batches its messages, and a partial update sent close to the create gets dropped. Traces were arriving with no end time, no output, and no outcome tags.

I caught it by reading traces back through the API instead of trusting the write. Now every span carries its end time at creation and the trace is re-sent whole as an upsert.

All three were invisible from the logs. That's the pattern: verify by reading the result back, not by watching for errors.

PART 5 — The Opik module

Screen: `opik_integration.py`, scrolled to the imports.

Here's the proof it's standalone. Standard library, and `opik`. Nothing from this project. It talks to the LiveKit session by duck typing rather than importing LiveKit types, so this one file drops into any LiveKit Agents codebase.

Scroll to `astart`.

One detail that matters. Opening the trace is a synchronous HTTP round trip, and calling it from the agent entrypoint stalled the audio loop for about a second. LiveKit flagged it. It runs on a worker thread now, and there's a test that fails if it ever goes back on the event loop. Observability must never degrade the call it's observing.

Scroll to `default_evaluators`.

Five evaluators run on every call. Two are deterministic: did it book, and did it confirm who it was speaking to before disclosing any health value. Those are facts, so a language model would only add noise.

Three are LLM judges. Biomarker fidelity is the important one: it gets the patient's real record as ground truth and checks every number the agent actually said. Then medical safety, looking for diagnosis or prescribing. Then conversation quality.

There are also two server-side rules created inside Opik itself, so evaluation runs on new traces automatically.

PART 6 — Live demo

Stop sharing the editor. Show two terminals and your phone.

Let me run it.

Point at Terminal 1, already running `python agent.py dev`.

The worker is running and registered with LiveKit.

Terminal 2.

```
python dispatch_call.py --patient PT-10432 --phone +91XXXXXXXXXX
```

Show the phone ringing. Answer on speaker.

Have the conversation. Confirm you're Aditya, let it explain the HbA1c, accept a time, let it read the confirmation number back, say goodbye.

Switch to Terminal 1, scroll to the post-call output.

There's the pipeline: Deepgram transcribing, OpenAI for the conversation, ElevenLabs for the voice. Then the recording saved, the analysis, the outcome, and the Opik trace finalised.

PART 7 — The trace, and proof the evaluation works

Screen: Opik, open the trace from the call you just made.

The input is the patient record and the call variables. The output is the outcome, whether an appointment was booked, and the confirmation details.

Play the audio attachment.

That's the real call recording, attached to the trace.

Expand the spans, then point at the feedback scores.

A conversation span with one child per turn. A tool span per function call with its arguments and result. The analysis. The evaluation. And the scores, each with the judge's written reasoning, so a low score is explainable rather than just a number.

Terminal: run the hallucination scenario.

```
python demo_pipeline.py --scenario hallucination
```

Wait for the scores.

This is a seeded bad call where the agent invents a normal HbA1c and suggests stopping a medication. Biomarker fidelity scores near zero, medical safety scores zero. On the clean call both score one. So the evaluation discriminates rather than rubber-stamping.

Conversation quality stays high here, and that's correct. That judge scores mechanics only. Accuracy is biomarker fidelity's job. If every judge scored everything, the scores would correlate and you'd lose the signal telling you which thing broke.

PART 8 — Why these services

Screen: back to the editor, or just talk.

Quickly, the technology choices.

Deepgram for transcription. This started as ElevenLabs for both speech stages. Then I measured a real call: transcription took two point seven seconds a turn while speech synthesis took zero point two eight. Transcription was two thirds of the lag. Deepgram is built for streaming, and `nova-3-medical` already knows clinical vocabulary.

ElevenLabs for the voice, because it sounds best and has genuine Indian English voices, which matters when your patients are called Aditya and Meera.

OpenAI for the conversation. I had Groq, which is very fast per token, but the models it serves reason before answering and on a phone call that thinking is heard as silence. `gpt-4o-mini` starts speaking immediately. Time to first token beats throughput here.

Groq is still used, for the post-call analysis and the judges. Those run after the call, so its latency costs nothing there.

Twilio for the number and SIP trunk, because LiveKit doesn't sell phone numbers.

PART 9 — Closing

Talk to camera.

Two honest things to finish.

The pattern that mattered most was verification. All three bugs I showed you were invisible from the logs and were found by reading the result back and checking it.

And what I'd do differently: change one variable at a time. Chasing that latency I switched the transcription model, overrode the endpointing config and enabled preemptive generation all at once. It broke the conversation, and because I'd changed three things I couldn't tell which one did it. I reverted and reintroduced them one by one.

With more time I'd add voicemail retry, consent and do-not-call handling, and compare evaluation scores across prompt versions in Opik, which is what would let you improve the agent systematically rather than by feel.

Thanks for watching. Happy to go deeper on any part of it.

Appendix — keep visible while recording

Stage	Choice	Reason in one line
Speech to text	Deepgram nova-3-medical	Streaming, 25ms endpointing, clinical vocabulary
Language model	OpenAI gpt-4o-mini	Answers immediately; reasoning models pause
Text to speech	ElevenLabs, voice Chandni	Best quality, genuine Indian English
Post-call analysis	OpenAI, strict JSON schema	Validated object, not parsed prose
Judges	Groq gpt-oss-120b	Off the call path, so latency is free
Telephony	Twilio SIP trunk via LiveKit	LiveKit does not sell numbers

Numbers to quote: 47 tests. About 4,700 lines of Python. Five evaluators plus two server-side rules in Opik. Transcription 2.69s a turn on the old provider against 0.28s for speech.

If you run long, cut Part 8 to just Deepgram and OpenAI, and cut the [data/patients.json](#) paragraph at the end of Part 3.